

为什么需要状态机？状态机让系统能明确知道"每个任务现在处于什么阶段"，避免出现"任务已经结束了但还在给它发消息"或"任务还在运行但资源被回收了"这类问题。

七种任务类型：

任务类型	图标	说明	例子
<code>`local_bash`</code>	终端	本地 shell 命令	<code>`npm install`</code> 、 <code>`git status`</code>
<code>`local_agent`</code>	助手	本地子助手	让一个子助手去读文件
<code>`remote_agent`</code>	远程	远程助手（Bridge 模式）	通过 IDE 插件执行
<code>`in_process_teammate`</code>	队友	同进程队友	和另一个 AI 协作
<code>`local_workflow`</code>	工作流	工作流脚本	预设的自动化流程
<code>`monitor_mcp`</code>	监控	MCP 服务器监控	监控外部服务状态
<code>`dream`</code>	梦境	后台推测	预判用户下一步操作

安全设计：每个任务都有一个加密随机 ID ($36^8 \approx 2.8$ 万亿种可能)，防止恶意程序伪造任务 ID 来读取任务输出。

2.3 为什么需要检查任务状态？

任务在执行时，会持续产生输出（比如命令的 stdout、子助手的中间结果）。这些输出存在磁盘上，但主程序需要定期查看是否有新内容。

事件驱动 vs 定时轮询：



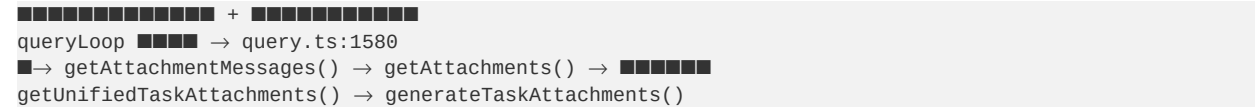
Claude Code 选择事件驱动模式，原因如下：

- 任务可能是在隔离环境中运行的，无法直接通知主程序
- 在工具循环的每次迭代中顺带检查，实现简单可靠（不需要维护定时器或长连接）
- 每次工具循环迭代之间最多 1 秒的延迟，对大多数任务来说可以接受
- 注意：pollTasks() 函数虽然定义了（含 POLL_INTERVAL_MS = 1000 常量），但实际没有被任何代码调用——真正的检查是通过 generateTaskAttachments() 在工具循环和用户输入处理时触发的

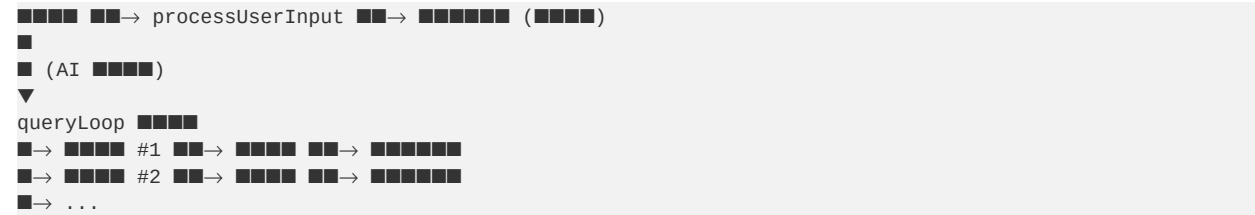
2.4 任务状态检查机制

重要纠正：pollTasks()

函数虽然定义了（src/utils/task/framework.ts:255），但从未被任何代码调用。真正被调用的是 generateTaskAttachments()，它通过事件驱动的方式触发：



任务状态检查不是定时轮询，而是事件驱动的——每次工具循环迭代或用户输入时顺带检查一次：




```

529 ( ) 400 Bad Request
  → 3
404 Not Found
429 ( )
  →
Enterprise
5xx ( )
408/409 ( / )
ECONNRESET/EPIPE
  →
401
  →
403
  →
400
  → max_tokens

```

> ※ 529 切换到备用模型有条件：仅当设置了 FALLBACK_FOR_ALL_PRIMARY_MODELS 环境变量，或者用户不是 Claude AI 订阅者且使用的是非自定义 Opus 模型时才会触发模型回退。订阅用户（非 Enterprise）遇到 529 不会触发自动回退。

3.4 持久无人值守模式

对于长时间运行的后台任务（比如通宵重构代码），有专门的持久重试模式：

```

10
32 5
30

```

为什么需要心跳？

- 如果 API 在长达几分钟的退避等待中没有响应，宿主环境（比如 IDE 插件）可能会认为会话已经“死”了，从而关闭连接
- 每 30 秒发送一个 api_retry 系统消息，告诉宿主：“我还活着，只是在等 API 恢复”

为什么持久模式退避上限是 5 分钟？

- 对于过载类错误（529），API 通常几分钟内就能恢复
- 5 分钟是一个合理的平衡点：既不会太频繁地打扰 API，也不会让用户等太久

3.5 模型回退

```

3 529
▼
FALLBACK_FOR_ALL_
PRIMARY_MODELS ?
! && Opus?
▼ ▼

```


[illegible]

为什么需要缓冲区 (13,000 tokens) ?

- 压缩需要时间来执行，期间对话还在继续
- 缓冲区确保压缩过程中不会出现“溢出”的尴尬情况
- 13,000 tokens 约等于 10,000 个英文单词，足够压缩期间使用

为什么需要断路器（连续 3 次失败后停止）？

- 如果压缩连续失败，说明压缩本身有问题（比如格式不对）
- 继续尝试只会浪费 API 调用次数和用户的钱
- 数据显示每天约 25 万次压缩调用，断路器能节省大量无效调用

4.3 会话内存压缩

除了传统的压缩方式，还有一种更智能的“会话内存”压缩：

```

#####
#####
# "####A####B# # #####
# #####C####D"# # #####
# # #
# → ##### # #####
# → ##### # #####
# # # auth.ts #
# # #####
# # #
# # # → #####
#####

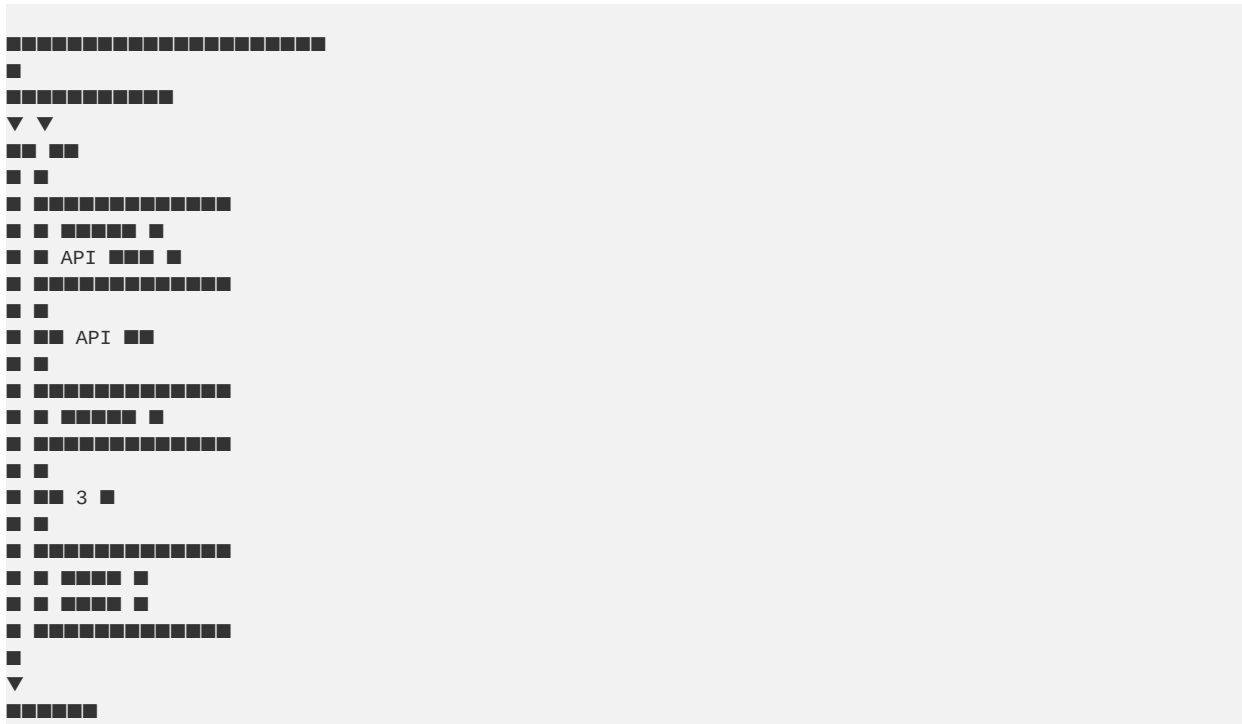
```

会话内存的好处:

- 由后台助手自动维护，不需要用户手动操作
- 内容是结构化的，保留关键信息
- 压缩时保留笔记，只丢弃无关细节
- 如果会话内存不可用，优雅回退到传统压缩

4.4 PTL (Prompt Too Long) 重试

如果不幸真的遇到了"提示太长"错误, 还有专门的恢复机制:



5. Agent 编排与子任务管理

5.1 为什么需要 Agent 编排?

当任务复杂时，一个 AI 助手可能忙不过来。比如：

- > 用户: "帮我分析这个项目的性能瓶颈"
- >
- > 如果只有一个助手，需要：
 - > 1. 读所有源码 → 耗时
 - > 2. 运行性能测试 → 不能同时做
 - > 3. 分析结果 → 要等前两步
- >
- > 如果有多个助手并行：
 - > 助手A: 读前端代码
 - > 助手B: 读后端代码
 - > 助手C: 运行性能测试
 - > 助手D: 检查数据库查询
- >
- > 然后主助手汇总所有结果 → 快得多！

5.2 三层架构




```

    ( )
    .....
    AbortController ( )
    readFileState ( )
    toolDecisions ( )
    systemPrompt ( )
    .....
    createChildAbortController()
    createSubagentContext()
    ▼
    ( )
    .....
    AbortController ( ) ← 
    readFileState ( ) ← 
    toolDecisions ( ) ← 
    systemPrompt ( ) ← 
    shouldAvoidPermissionPrompts = true 
    → 
    .....

```

为什么需要隔离？

- 防止污染：工作线程读文件时出错，不应该影响主助手的状态
- 防止阻塞：工作线程卡住了，主助手还能继续工作
- 防止权限冲突：工作线程在后台运行，不应该弹出对话框让用户确认

子 AbortController 的妙用：

- 父级中止 → 子级自动中止（级联取消）
- 子级中止 → 不影响父级（单个任务失败不影响整体）
- 使用 WeakRef 防止内存泄漏（不会因为忘记清理引用导致内存占用）

5.5 工作线程的完整生命周期

```

    ▼
    .....
    AgentTool 
    agentId ( "a3f8b2c1" ) 
    subagent_type 
    .....
    registerAsyncAgent() 
    AppState.tasks 
    AbortController 
    pending → running 
    .....
    runAgent() 
    .....
    query() → LLM 
    30 
    .....
    ▼ ▼
    ▼ ▼
    ▼ ▼

```



```
→ [REDACTED] → [REDACTED]
[REDACTED] (sequential.ts)
[REDACTED] → [REDACTED] → [REDACTED] → [REDACTED]
→ [REDACTED] → [REDACTED]
→ [REDACTED]
```

6. 错误处理与恢复机制

6.1 为什么需要分层错误处理？

不同的错误需要不同的处理方式：

错误类型 | 严重程度 | 处理方式

用户按了 Ctrl+C | 低 | 静默忽略，不记录日志

命令执行失败（如 `rm` 不存在的文件） | 低 | 记录 debug 日志，继续执行

磁盘空间不足 | 中 | 尝试清理，提示用户

API 认证过期 | 高 | 刷新令牌，重试请求

代码崩溃 | 严重 | 记录诊断信息，优雅退出

分层处理的好处：

- 不把用户当专家：普通错误给简单提示，复杂错误给详细信息
- 不浪费日志空间：低级别错误不记录到 error 日志
- 不掩盖问题：严重错误一定会被注意到

6.2 25+ 错误场景分类

```
API [REDACTED]
[REDACTED]
[REDACTED] classifyAPIError() [REDACTED]
[REDACTED] aborted [REDACTED]/[REDACTED]
[REDACTED] api_timeout API [REDACTED]
[REDACTED] repeated_529 [REDACTED]
[REDACTED] capacity_off_switch [REDACTED]
[REDACTED] rate_limit [REDACTED]
[REDACTED] server_overload [REDACTED]
[REDACTED] prompt_too_long [REDACTED]
[REDACTED] pdf_too_large PDF [REDACTED]
[REDACTED] pdf_password_protected PDF [REDACTED]
[REDACTED] image_too_large [REDACTED]
[REDACTED] tool_use_mismatch [REDACTED]
[REDACTED] auth_error [REDACTED]
[REDACTED] invalid_api_key API [REDACTED]
[REDACTED] token_revoked [REDACTED]
[REDACTED] credit_balance_low [REDACTED]
[REDACTED] invalid_model [REDACTED]
[REDACTED] server_error [REDACTED]
[REDACTED] connection_error [REDACTED]
[REDACTED] ssl_cert_error SSL [REDACTED]
[REDACTED] unknown [REDACTED]
[REDACTED]
```

6.3 工具执行时的错误处理

当 AI 调用某个工具（比如读文件、写文件）时，工具可能出错：

```
[REDACTED]
[REDACTED]
▼
[REDACTED]
[REDACTED] AbortError? [REDACTED]
[REDACTED] ([REDACTED]) [REDACTED]
```

关键设计：工具错误不会导致程序崩溃。错误被封装成消息返回给 AI，由 AI 决定是重试还是放弃。这就像“上级把问题抛回给当事人自己解决”。

6.4 对话恢复（崩溃后续传）

如果程序崩溃了，用户重新启动时，需要能接着之前的地方继续：

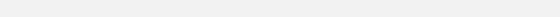
[illegible]

为什么需要这么多过滤步骤？

- 流式输出可能在任何位置被截断
- 不完整的信息（比如只有 `tool_use` 没有 `tool_result`）会导致 API 错误
- 这些过滤步骤确保恢复后的消息是“干净”的，不会触发新的错误

6.5 会话状态恢复

当用户用 `--resume` 参数恢复会话时，系统会重建所有状态：



Worktree 恢复 | 使用 `process.chdir()` 作为原子存在性检查
任务注册替换 | 替换时携带现有状态向前，不丢失已有数据

9.4 竞态防护

什么是竞态？多个线程/进程同时操作同一份数据，导致结果不可预测。

```
[X] ■■■■
■■■A■■■ count = 5
■■■B■■■ count = 5
■■■A■■■ count = 6 (5+1)
■■■B■■■ count = 6 (5+1■■■■A■■■)
■■■count = 6■■■■ 7

[OK] ■■■■
■■■A■■■■ count += 1 → count = 6
■■■B■■■■ count += 1 → count = 7
■■■count = 7
```

Claude Code 中的竞态防护：

- 机制 | 说明
- **任务通知幂等性** | 原子性检查 + 设置 `notified` 标志，防止重复通知
- **顺序执行包装器** | `sequential.ts` 确保并发调用逐个执行
- **任务输出增量补丁** | 仅偏移量补丁，避免覆盖状态转换
- **子 AbortController** | `WeakRef` 保护，防止内存泄漏

10. 关键文件索引

以下是报告中涉及的所有关键源文件，按功能域分组：

任务管理

- 文件 | 内容 | 行数（估计）
- `src/Task.ts` | 任务类型定义、状态机、ID 生成 | ~200
- `src/utils/task/framework.ts` | 任务注册、通知驱逐、事件驱动检查 | ~400
- `src/utils/task/diskOutput.ts` | 磁盘持久化、5GB 上限、安全文件打开 | ~300
- `src/tasks/stopTask.ts` | 类型化任务停止操作 | ~100
- `src/tasks.ts` | 任务类型注册表 | ~50

API 通信

- 文件 | 内容 | 行数（估计）
- `src/services/api/withRetry.ts` | 重试引擎、退避算法、回退、心跳 | ~500
- `src/services/api/errors.ts` | 25+ 错误场景分类、用户提示 | ~300
- `src/services/api/errorUtils.ts` | 连接错误提取、SSL 检测 | ~150

上下文管理

- 文件 | 内容 | 行数（估计）
- `src/services/compact/autoCompact.ts` | 自动压缩、断路器（3 次失败停） | ~200
- `src/services/compact/compact.ts` | PTL 重试循环 | ~300
- `src/services/compact/sessionMemoryCompact.ts` | 基于会话内存的压缩 | ~200
- `src/services/SessionMemory/sessionMemory.ts` | 后台知识提取（非阻塞） | ~300

Agent 编排

文件 | 内容 | 行数 (估计)

`src/coordinator/coordinatorMode.ts` | 多 agent 编排 (四阶段方法) | ~400
`src/utls/forkedAgent.ts` | 隔离子 agent 执行 | ~300
`src/tools/AgentTool/AgentTool.tsx` | 核心编排原语 | ~1900
`src/tools/AgentTool/runAgent.ts` | 核心 agent 查询循环 | ~500
`src/tools/AgentTool/resumeAgent.ts` | 从磁盘恢复已停止 agent | ~300
`src/tools/SendMessageTool/SendMessageTool.ts` | 继续已停止 agent | ~400
`src/tools/TaskStopTool/TaskStopTool.ts` | 终止 agent | ~100
`src/services/AgentSummary/agentSummary.ts` | 30 秒进度摘要 | ~200
`src/utls/messageQueueManager.ts` | 通知投递管理 | ~200
`src/utls/queueProcessor.ts` | 命令队列处理 | ~200

错误处理与恢复

文件 | 内容 | 行数 (估计)

`src/utls/sessionRestore.ts` | 完整恢复引擎 | ~550
`src/utls/sessionState.ts` | 状态机 (idle/running/requires_action) | ~100
`src/utls/conversationRecovery.ts` | 中断检测、工具过滤、状态恢复 | ~300
`src/utls/sessionStorage.ts` | JSONL 持久化 | ~200
`src/utls/errorLogSink.ts` | 缓冲 JSONL 写入器 (1s 刷新) | ~150
`src/utls/errors.ts` | 自定义错误类 | ~100
`src/services/tools/toolExecution.ts` | 工具执行错误处理 | ~200

进程级保障

文件 | 内容 | 行数 (估计)

`src/utls/gracefulShutdown.ts` | 信号处理、分阶段关闭、故障安全 | ~300
`src/services/preventSleep.ts` | Mac caffeinate 管理器 (5min 自愈) | ~100
`src/utls/cleanup.ts` | 磁盘 GC (30 天轮换) | ~300
`src/utls/cleanupRegistry.ts` | 关闭钩子注册 | ~50
`src/utls/sequential.ts` | 防竞态队列序列化 | ~50
`src/utls/abortController.ts` | 子 AbortController 工厂 | ~100
`src/utls/combinedAbortSignal.ts` | 超时 + 信号组合 | ~50
`src/utls/sleep.ts` | 可中断睡眠 | ~30
`src/utls/timeouts.ts` | 超时配置 | ~50

安全与权限

文件 | 内容 | 行数 (估计)

`src/hooks/toolPermission/` | 权限状态机 | ~500
`src/utls/secureStorage/fallbackStorage.ts` | keychain → 纯文本回退 | ~100
`src/hooks/useCancelRequest.ts` | 用户取消 (Escape/Ctrl+C) | ~100
`src/utls/execFileNoThrow.ts` | 永不抛出的 exec 包装 | ~50
`src/utls/process.ts` | EPIPE 处理 | ~50

附录：核心数据流图

子 Agent 完整生命周期

